Logical Structured Context Worlds: Exact Context Boundaries Without a Runtime

Declarative SCW boundaries over Maxey0's five typed memory tiers, with user-side ChatGPT advertising privacy as a first application, tested on a real ChatGPT data export

By Mike Cohan (Human0) and Maxey0-Claude (Opus 5.5) · Maxey0 — AI Systems Architecture and Research

human@maxey0.com · 1 October 2026 · Maxey0 project: SCW Privacy · research paper with a working application (version 3.1)

Abstract

A Maxey0 Structured Context World (SCW) is a bounded, addressable context space with explicit rules for what may enter and leave it. Where the owner of a boundary does not control the runtime (inside another vendor's product, between tenants sharing a context store, or between agents sharing one model context), the boundary cannot be a process or a sandbox. This paper defines the **logical SCW**: an SCW whose boundary is a declarative policy object, enforced by one pure evaluator on every egress path, dominated by a fixed set of invariants that no configurable field can weaken, and whose evaluator must name the rule behind every withheld signal. We pair the pattern with Maxey0's memory model of **five typed tiers** (Working, Episodic, Semantic, Persistent, Procedural) joined by an evidence-gated promotion gate, so that whole kinds of memory can be removed from an egress path by construction rather than by a setting. As a first application we build a user-side advertising boundary for ChatGPT Free and Go users. Their ad controls today act on the whole account, on one ad at a time, or on one Temporary Chat, which shows no ads [1]; none lets a user say which kinds of conversation may inform ads. The application, *Maxey0 SCWs for ChatGPT Ads* (version 3.1), is one self-contained React file with no server, no storage and no user data. A seeded property test of 20,000 random profiles, worlds, messages and histories found zero violations of 23 per-case checks, and each of 13 deliberately injected faults was caught. Replaying one real export of 602 chats (9,295 user messages) through the evaluator on its owner's computer produced zero invariant breaches in all 18 runs (three presets, three worlds, with and without stated interests) and suppressed, in every world, each of the 43 chats that its keyword classifier flagged as sensitive. A logical SCW is exactly as strong as the guarantee that all egress passes through its evaluator. In the ChatGPT application it cannot bind OpenAI's ad system, so it gives the user an exact, inspectable account of their own policy, points to the controls OpenAI does enforce, and labels everything else as advisory.

Keywords: Structured Context World, SCW, logical SCW, logical isolation, declarative boundaries, multi-agent context, conversational advertising, ChatGPT ads, privacy controls, typed memory, promotion gate, data minimization, information-flow policy, contextual integrity, user-side privacy tooling, Maxey0

Contents

1 Introduction: context boundaries without a runtime · **2** Background, earlier versions and related work · **3** The logical SCW · **4** Memory: five typed tiers and a promotion gate · **5** What a ChatGPT export holds, by tier · **6** The application: Maxey0 SCWs for ChatGPT Ads · **7** Evaluation · **8** Threat model and limitations · **9** Discussion and conclusion · Author statement · Note on sources and claims · References · Appendix A: export schema observed · Appendix B: reproducing the results

1 Introduction: context boundaries without a runtime

Many context boundaries must be drawn where their owner does not control the runtime: inside another vendor's product, between tenants that share a context store, or between agents that share one model context. The Maxey0 architecture calls the unit such a boundary encloses a Structured Context World (SCW) [5]: an explicit, addressable context space with rules for what may enter and leave it.

Advertising in a chat assistant is a sharp instance. An assistant that carries ads reads the same context the user writes into. OpenAI's ad selection uses the current chat and basic context such as general location and language; when ad personalization is on, it may also use how the user interacts with ads, and past chats and memory [1]. The user-facing controls are coarse. A person who is happy for shopping questions to shape their ads may not want a work conversation, a study session or a question about a medication to do the same. What such a person needs is a boundary per kind of conversation, with a precise statement of what crosses it.

A user cannot place a process boundary inside someone else's product. They can, however, state a policy exactly, compute its consequences on their own data, and carry the result to the switches that do bind the provider. This paper calls an SCW built that way a **logical SCW**. Its boundary is an object, not a runtime; its enforcement is a function, not a sandbox.

> **The claim of this paper**
>
> A context boundary can be made exact without being physical, provided that (a) it is written as a declarative object, (b) one pure evaluator decides every egress, (c) a small set of invariants dominates every configurable field, and (d) the evaluator explains every withheld signal. The boundary is then exactly as strong as the guarantee that all egress passes through the evaluator, and no stronger.

Relative to version 2.3, this paper (version 3.1) makes four changes. It replaces a four-level memory stack with Maxey0's five typed tiers and a promotion gate (Section 4). It rebuilds the application [4] around that model, with visualizations that let a non-specialist see the boundary (Section 6). It closes eight defects that an independent review found in version 3, among them an identity leak through stated interests and a contextual mode that behaved like open (Section 2.3). And it re-measures everything with the corrected evaluator, including a fresh replay of one real ChatGPT export (Section 7).

2 Background, earlier versions and related work

2.1 Structured Context Worlds and SCW0

In the Maxey0 execution-space model, an SCW is a bounded semantic and contextual space that binds context, identity, capabilities, resources, provenance, admission rules and permitted transitions into one addressable unit. SCW0, the root world, creates SCWs, routes context between them and evaluates boundary transitions [5]. The System One paper separates decision from authorization, execution and observation [6]. The logical SCW in this paper is a small, concrete instance of both ideas: the policy object is the authorization boundary, and the withheld-signal ledger is the observation record.

2.2 What OpenAI documents

OpenAI's Help Center states that ads may appear on the Free and Go plans and not on Plus, Pro, Business, Enterprise or Edu; that accounts identified as belonging to users under 18 do not see ads; that advertisers receive only aggregated, non-identifying performance information such as views or clicks; and that ads are not eligible to appear near sensitive or regulated topics, including personal health, mental health or politics [1]. Its controls act at three levels. For the whole account, a user can turn off ad personalization, stop past chats and memory from informing ads, and clear ads data; for one ad, a user can hide it, report it and see why it was shown ("About this ad"); and a Temporary Chat shows no ads [1]. A Temporary Chat also does not create or update memories and stays out of chat history [2]. The data export is requested from Settings, Data controls, and arrives as a zip file [3].

If advertisers see only aggregates, the privacy-relevant boundary is not between the user and the advertiser. It is between the user's context and the *ad-selection system* that reads it. The application therefore models what the ad system may read, and the evaluation reports what would cross each world toward it.

2.3 Earlier versions

The first version of the application, built by the Maxey1 agent, set out the user-facing idea: privacy gates for kinds of conversation, each with ad limits, category allowances, data-sharing switches and a list of accessible memory tiers, plus a JSON profile to paste into ChatGPT. It had three problems. Its tiers (Scratchpad, Episodic, Persistent, Permanent) were a duration stack rather than kinds of memory. Its privacy score was a sum of fixed points for switches. And it claimed an enforcement it could not deliver: it told users to export their configuration and "provide it to ChatGPT to establish privacy boundaries that advertisers must respect." Versions 2 through 2.3 introduced the evaluator, the invariants and an in-page export audit, but kept a four-level tier ceiling (L1 to L4). Version 3 adopted the five typed tiers and stated, on every relevant screen, which channel is enforced and which is advisory.

An independent review of version 3, by separate agents that had not seen the work produced, found eight defects, all fixed in version 3.1. An interest containing an email address could be released (an INV-1 breach). A blocked topic could still be released from an earlier turn or an interest. Contextual and open worlds behaved identically. A spent session budget still released labels. A blocked topic was reported as a sensitive suppression. The Latent Map used its own copy of the per-tier rule and ignored a global switch. The Advertiser Lens showed a request for turns that make none. And importing a profile with no SCWs crashed the Lens. Section 7.1

reports how often the first four occurred in version 3.

2.4 Related work

Logical SCWs compose established mechanisms rather than inventing new ones. Information-flow control labels data and checks every flow against a lattice of security classes [7]; the decentralized label model lets each owner attach its own policy to shared data [8]. Contextual integrity holds that a flow of information is appropriate when it matches the norms of the context it came from [9]; an SCW makes that context explicit and its norms executable. Privacy-preserving advertising systems such as Adnostic [10] and Privad [11] moved targeting onto the user's device so that profiles never leave it.

The application differs from all of these in where it sits. It does not change how ads are selected. It computes, on the user's own data, exactly what a stated policy would let an ad-selection system read, explains every signal it withholds, and states which parts of the policy the provider enforces. The contribution is that composition, applied as a user-side logical SCW over typed memory.

3 The logical SCW

A logical SCW is a tuple of declarative fields. A separate, global set of invariants applies to every SCW, and no SCW field can weaken it (Figure 1, Tables 1 and 2).

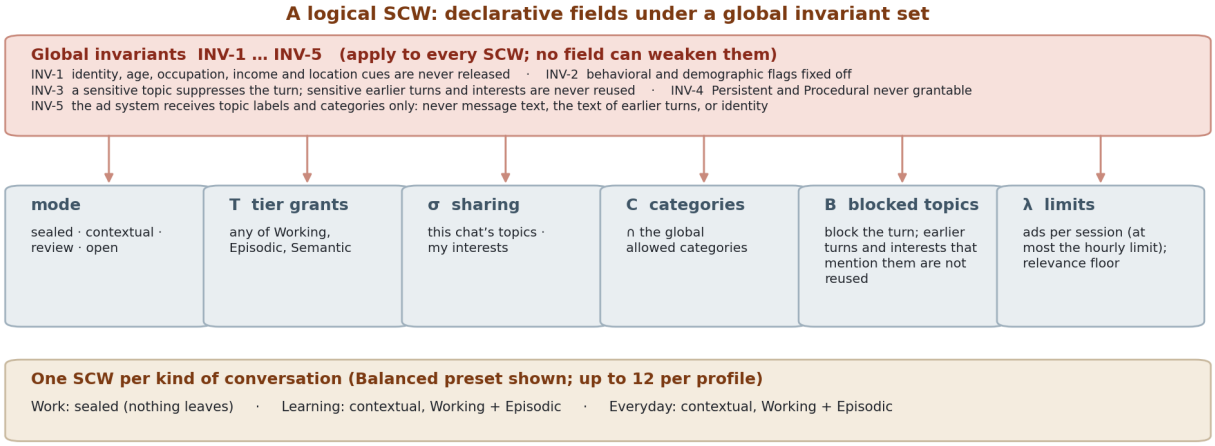


Figure 1. The logical SCW object: six configurable fields per world, dominated by one global invariant set.

$$\mathbf{SCW = (\ id,\ mode,\ T,\ \sigma,\ C,\ B,\ \lambda\ )\ under\ INV = \{\ INV\text{-}1\ \ldots\ INV\text{-}5\ \}}$$

Table 1. The fields of a logical SCW.

Field	Meaning	Values in the application
mode	How the world treats egress	sealed (nothing leaves); contextual (topic labels from this chat only, never the user's interests); review (as open, but a matched ad is held for approval); open (every granted tier)
T tier grants	Which memory tiers the ad system may read in this world	any subset of Working, Episodic, Semantic. Persistent and Procedural cannot be granted.
σ sharing	Which sources may be used	this chat's topics (Working, Episodic); stated interests (Semantic, open and review worlds only). Each tier is also gated by a profile-wide switch.
C, B	Allowed categories; blocked topics	four ad categories, intersected with the profile's; free-text topics, matched as whole words (single words) or substrings (phrases) in the current message, earlier turns and interests
λ limits	Per-world ad count and relevance floor	combined with the profile-wide hourly limit, which the evaluator applies per session because it has no clock, and the profile-wide floor; the higher floor applies

Table 2. The global invariants. The wording follows the application's, in the third person; for INV-2 and INV-4 it adds how each is enforced.

Invariant	Statement
`INV-1`	Identity cues (email, phone, name) and age, occupation, income and location cues are never released from a message or an interest, in any SCW.
`INV-2`	Demographic and behavioral data are never released; the flags are fixed to off and cannot be set by editing or import.
`INV-3`	A sensitive topic (health, mental health, politics, religion, sexuality, financial hardship) suppresses the whole turn: nothing is released and no ad is shown. Sensitive earlier turns and interests are never reused.
`INV-4`	Persistent and Procedural memory can never be granted to an SCW; grant lists are normalized on every edit and every import.
`INV-5`	The ad system receives topic labels and their categories only (vocabulary words or the user's bounded interests): never message text, the text of earlier turns, or identity.

Tier grants are a set rather than a ceiling on purpose. The tiers are kinds of memory, not levels of one quantity, so there is no order in which reading Semantic should imply reading Episodic. Instead, exposure is made explicit by weights (Working 0.2, Episodic 0.3, Semantic 0.5) that the application shows alongside every world, and the two tiers whose contents identify or instruct the person are removed from the grantable set altogether.

3.1 Signals, not text

The evaluator never releases message text. Each input is first reduced by a deterministic keyword classifier to *labels* from a fixed vocabulary (such as `kubernetes` or `trail`), each with an ad category, plus an optional sensitive class and a list of identity-cue kinds. A released signal is a label, its category and the tier it came from. The most the ad system could ever receive is a vocabulary label, or an interest the user typed that the classifier could place in an allowed category. An interest is at most 40 characters and four words and carries no identity cue; this is enforced on entry, on import and again by the evaluator.

3.2 The evaluator and its rule order

The evaluator takes the profile, one SCW, the current message (Working), the earlier messages in the same SCW's session (Episodic), the number of ads already shown in that session, and the profile's stated interests (Semantic). Rules run in the fixed order of Figure 2, and the first rule that ends the turn sets its outcome (Table 3). Identity cues are always written to the ledger first. A sensitive topic then ends the turn as *suppressed*, a blocked topic as *blocked* and a sealed world as *sealed*; in each case nothing is released.

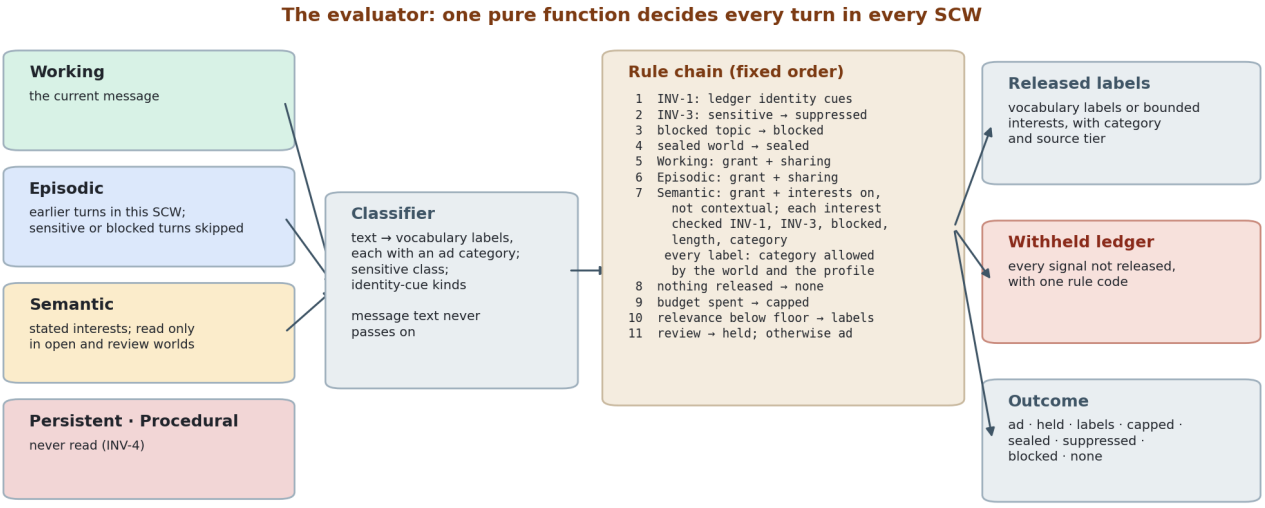


Figure 2. The evaluator. Three grantable tiers feed one fixed rule chain; the sealed tiers are never read. It returns released labels, a ledger of withheld signals with one rule code each, and one of eight outcomes.

Otherwise each tier is read only if it is granted and its sharing switches are on, and contextual worlds never read Semantic. Earlier turns that were sensitive or mention a blocked topic contribute nothing, and their labels are ledgered. Each interest is checked, in order, for an identity cue, a sensitive topic, a blocked topic, its length and its category before it is considered. Every label is released only if its category is allowed by both the world and the profile.

If nothing is released, the outcome is *none*. If the session's ad budget (the lower of the world's per-session limit and the profile's hourly limit) is already spent, no ad request is made, the labels move to the ledger and the outcome is *capped*. Otherwise ads are matched only against released labels, from a fictional inventory, skipping blocked brands. Relevance is 55 + 20 per matched label, plus 10 if a match came from the current message, capped at 100, and must reach the higher of the world's and the profile's floors. A turn whose best match falls below the floor ends as *labels*; a matched ad is *held* in a review world and served (*ad*) otherwise.

Table 3. The eight outcomes of one turn.

Outcome	When	What reaches the ad system
`ad`	an ad reached the relevance floor in an open or contextual world	released labels; one ad is shown
`held`	as ad, in a review world	released labels; the ad waits for the user's approval
`labels`	labels released, but no ad reached the floor	released labels; no ad is shown
`capped`	labels would be released, but the session budget is spent	nothing: no ad request is made
`sealed`	the world is sealed	nothing
`suppressed`	the current message is sensitive (INV-3)	nothing
`blocked`	the current message mentions a blocked topic	nothing
`none`	no granted tier yielded a label in an allowed category	nothing

Every withheld signal carries one rule code: INV-1, INV-3 or INV-4 for an invariant; BLOCK for a blocked topic, SEALED for a sealed world and BUDGET for a spent budget; NO-GRANT for a tier not granted; SHARE-CTX, SHARE-SESSION and SHARE-PREFS for a sharing switch that is off; CONTEXTUAL for interests in a contextual world; and CATEGORY, UNCLASSIFIED and LENGTH for labels and interests that fail the category or length checks. INV-2 and INV-5 never need a code, because they hold by construction: the behavioral and demographic flags are never read, and no text is ever a released signal.

3.3 Properties

Table 4. Properties of the evaluator and how each is checked (Section 7.1).

Property	Statement	How it is checked
Determinism	The same inputs always yield the same decision.	Property test (re-evaluation per case)
Invariant dominance	No released signal is from a sealed tier, identity-shaped or sensitive; a sensitive turn releases nothing; no setting of any field changes this.	Property test, 20,000 cases at two seeds; real-export replay
Blocked-topic containment	A blocked topic in the current message ends the turn; no label is released from an earlier turn or interest that mentions one.	Property test
Grant and sharing soundness	A label is released only from a granted tier whose sharing switches are on, and only in a category both the world and the profile allow.	Property test
Mode containment	Sealed releases nothing; contextual never reads Semantic; review never serves an unapproved ad.	Property test
Minimization	Released labels are vocabulary items or bounded user interests: no label longer than 40 characters or four words.	Property test, with an over-long interest in the input pool
Budget and ad soundness	Nothing is released once the session budget is spent; a served ad respects category, relevance floor and blocked brands.	Property test
Explainability	Every identity cue in the current message is ledgered as INV-1, and every label found in it is either released or ledgered with a rule code.	Property test (non-sensitive messages); visible in the simulator

4 Memory: five typed tiers and a promotion gate

Maxey0 memory is not a duration stack. It has five tiers typed by what they hold, and every move between tiers passes through one evidence-gated promotion gate (Figure 3).

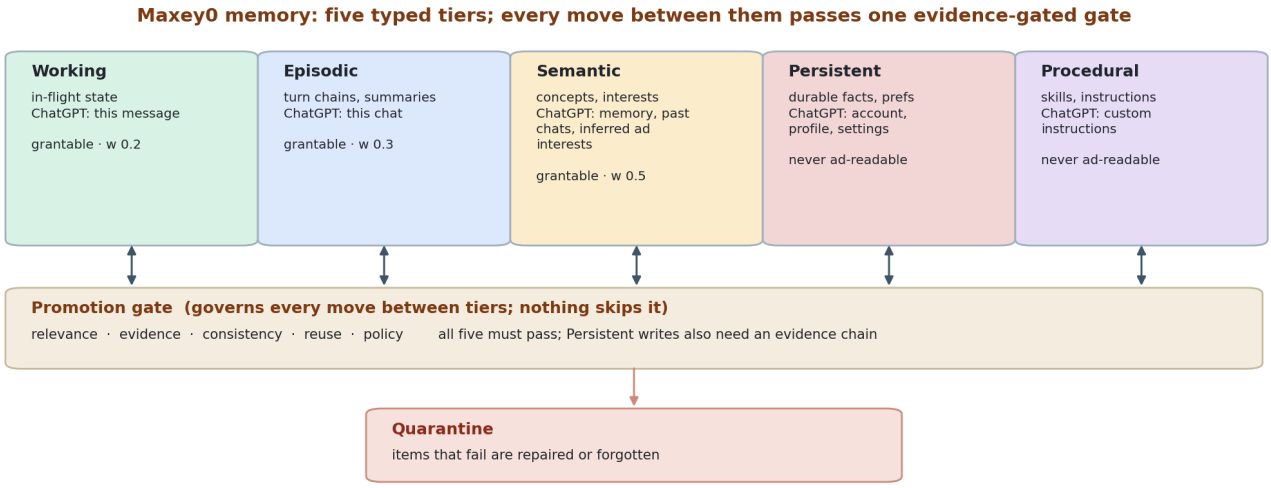


Figure 3. The five tiers, each with its ChatGPT counterpart and its ad-exposure weight, and the promotion gate with quarantine.

Table 5. The five tiers and how each maps onto ChatGPT.

Tier	Holds	Lifecycle	ChatGPT counterpart	Ad-readable
Working	volatile in-flight state	time-to-live (TTL) enforced; purged at session close	the message being written	if granted, with this chat's topics shared
Episodic	turn chains, session summaries	timestamped; summarized at session close	earlier turns in this chat	if granted, with this chat's and this session's topics shared
Semantic	concept graph: concepts, interests	admitted only through the gate	saved memory, what ChatGPT draws from past chats, inferred ad interests	if granted, in an open or review world, with interests and personalization on
Persistent	durable facts, versioned preferences	written rarely, with an evidence chain	account identity, profile, settings	never
Procedural	skills and instructions	creation, revision, deprecation	custom instructions; this profile	never

Table 5 maps each tier onto its ChatGPT counterpart. The promotion gate checks five things before an item moves between tiers: relevance, evidence, consistency, reuse and policy. Items that fail go to quarantine, where they are repaired or forgotten; nothing skips the gate. The model matters for advertising privacy because the Semantic tier is where consolidated facts about a person live, and it is the tier that ChatGPT's "past chats and memory" setting governs. Besides saved memory and what ChatGPT draws from past chats, Section 5 shows that a real export contains a third Semantic object: the interests the ad system itself inferred.

In the application, the Semantic tier is represented by the user's stated interests. The promotion gate is illustrated, not executed: the application never moves items between tiers on its own, and every interest is entered, and can be removed, by the user.

5 What a ChatGPT export holds, by tier

The first author requested a ChatGPT data export (the folder name is dated 11 September 2026) and supplied it as a dataset, on the condition that none of it be built into the application.

To learn what an export contains we read its *schema* (key names, value types, string lengths) and computed aggregate counts on the owner's computer, during the work on version 2 (29 September 2026). No message text, interest text, email, birth year or account ID was displayed to Maxey0-Claude or copied off the machine. The folder held 972 files at the top level, plus one subfolder, in 1.5 GB; ten of the files carry what this problem needs (Table 6 and Appendix A).

Table 6. The export files this problem needs, by tier.

File	Content (schema)	Tier
`user.json`	email, birth_year, chatgpt_plus_user, id	Persistent
`user_settings.json`	settings incl. training_allowed	Persistent
`ads.json: ads_profile`	inferred_profile.user_interests[] (4 items), profile_version	Semantic
`ads.json: ad_history, ad_hides, reported_ads`	event_name, event_time, advertiser_name, ad_title	Episodic
`conversations-000…006.json`	messages; memory_scope; is_do_not_remember	Episodic
`saved memories`	no file present	—

The conversation files are Episodic: they record turns. When ChatGPT draws on past chats to personalize a reply or an ad, it makes a Semantic use of them, which is why the “past chats and memory” switch belongs with the Semantic tier.

Three facts about this one export shaped the design. First, it contains the **ad system’s own inferred interest profile**: Semantic memory the user did not write and may never have seen. OpenAI states that deleted ads data is no longer used to show ads [1]; it does not say whether this inferred-interest list counts as ads data. Second, it contains **no saved-memory file**, and none of its 33,673 messages is addressed to or from the memory tool. Third, every chat carries a memory flag: all 602 had `memory_scope = global_enabled` and none was marked do-not-remember. The account-level files map onto Persistent memory, which is exactly the tier INV-4 removes from every advertising world.

6 The application: Maxey0 SCWs for ChatGPT Ads

The application [4] is one React file that imports only React and an icon library, carries its own styles, makes no network requests and stores nothing. Every advertiser in it is fictional. Table 7 lists its eight pages.

Table 7. The eight pages of the application.

Page	What the user does	What it demonstrates
Overview	Reads the privacy score; picks a preset; follows the exposure flow; opens any replay cell	The whole boundary at once: tiers and identity cues → worlds → advertiser or withheld, and 18 synthetic prompts × every world
Latent Map	Watches fictional advertisers probe the boundary; selects a world to edit it	The boundary as a live system: sealed core, tier rings, worlds, admitted and deflected probes
SCW Studio	Edits each world's mode, tiers, categories, sharing, limits and blocked topics	The control plane, with a tier × world grant grid and a live preview per world
Simulator	Chats inside a chosen world	Each rule with pass or fail, released and withheld signals with rule codes, a fictional ad, and the same message in every world
Advertiser Lens	Chooses a world and a prompt	The complete payload the ad system would receive, with user ID, message text, history, demographics and location all null; when a turn makes no request, the Lens says so
Memory	Sees the five tiers and the promotion gate (illustrated, not executed); edits interests	The typed memory model applied to the user's own Semantic tier
Learn SCWs	Reads five short lessons	Logical SCWs explained, each lesson linked to the page that shows it
Apply to ChatGPT	Sets global controls; reviews ChatGPT's settings; copies the instruction; exports or imports the profile	The split between enforced and advisory channels

6.1 Visualizations that teach the boundary

The application is meant to teach logical SCWs to a general audience, so each visualization is computed from the live policy rather than drawn as an illustration. Each either runs the evaluator or reads its per-tier rule, so a channel the evaluator keeps closed is never shown open. The **exposure flow** (Figure 4) links every tier and identity cues to every world, and each world to the advertiser or to withheld. Each band is one channel: solid bands into a world are open and dashed ones closed, and a world's band to the advertiser is as wide as its open channels.

The **replay grid** runs the 18 synthetic prompts through every world and marks each cell with its outcome in color and in text (AD, HOLD, TAG, CAP, SEAL, OFF, BLK, or a dash for none), with an invariant-breach counter that should always read zero. On the **Latent Map** (Figure 5), each probe asks whether an ad of its category could be served from a world's current message. It is admitted only if that world's Working channel is open, the category is allowed there and globally, and the brand is not blocked. The map does not model Episodic, Semantic, relevance, budget or the review hold, A change to a world's mode, Working grant or categories redraws its channels at once and changes which new probes pass within a few seconds.

6.2 Three channels, three levels of force

The **settings checklist** points to OpenAI's own controls: memory and chat-history reference, ad personalization, clearing ads data, and Temporary Chat for anything that belongs in a sealed world [1][2]. Only these change what OpenAI's systems read. The **instruction block** restates each world's rule, the invariants, categories, blocked brands and limits for the assistant. It asks the user to start each chat with "World: <name>" so that the assistant knows which rules apply, and to treat an unnamed chat by the strictest world's rules. It can shape what the assistant uses in conversation and cannot change ad selection.

The **profile file** (format 2.0.0, written by application version 3.1) is the user's portable copy of the policy. Import accepts profiles from earlier versions and converts their tier names; it drops any grant that would violate INV-4 and any interest that is over-long or carries an identity cue, rejects a profile with no SCWs, and clamps limits to the ranges the interface allows.

> **What the tool does not claim**
>
> It does not claim that ChatGPT will honor the user's SCWs. It shows exactly what the user's policy would release, points to the switches OpenAI enforces, and labels the pasted instruction as a request.

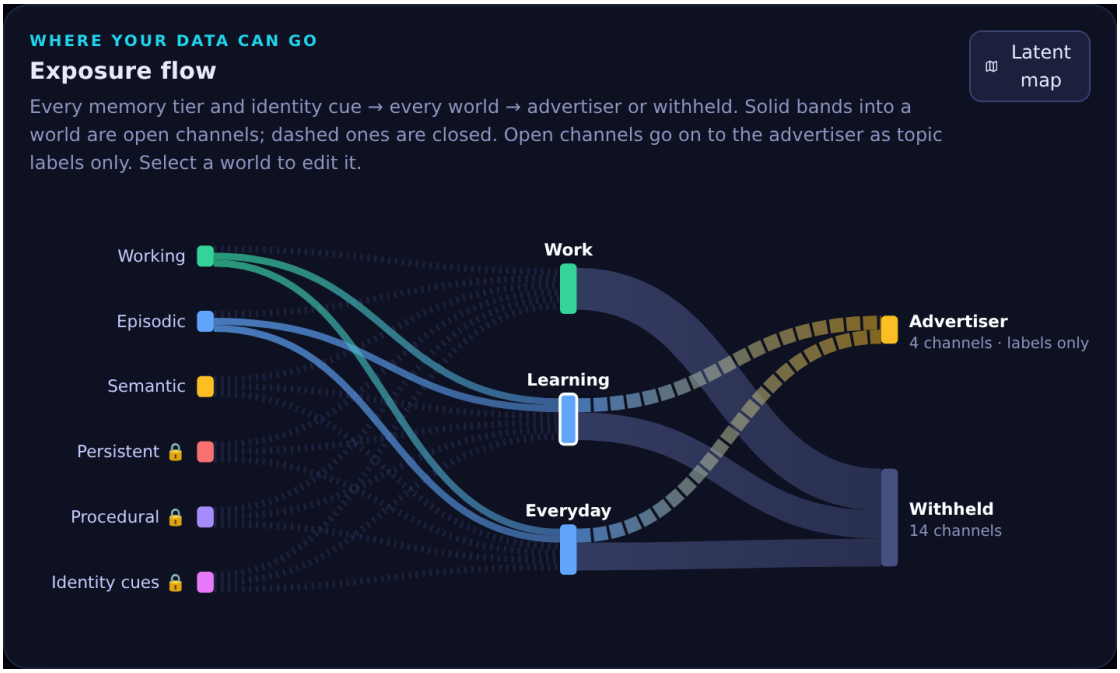


Figure 4. The exposure flow on the Overview (Balanced preset). Four channels are open: Working and Episodic into Learning and into Everyday. The other 14, including every channel from Persistent, Procedural and identity cues, end at Withheld.

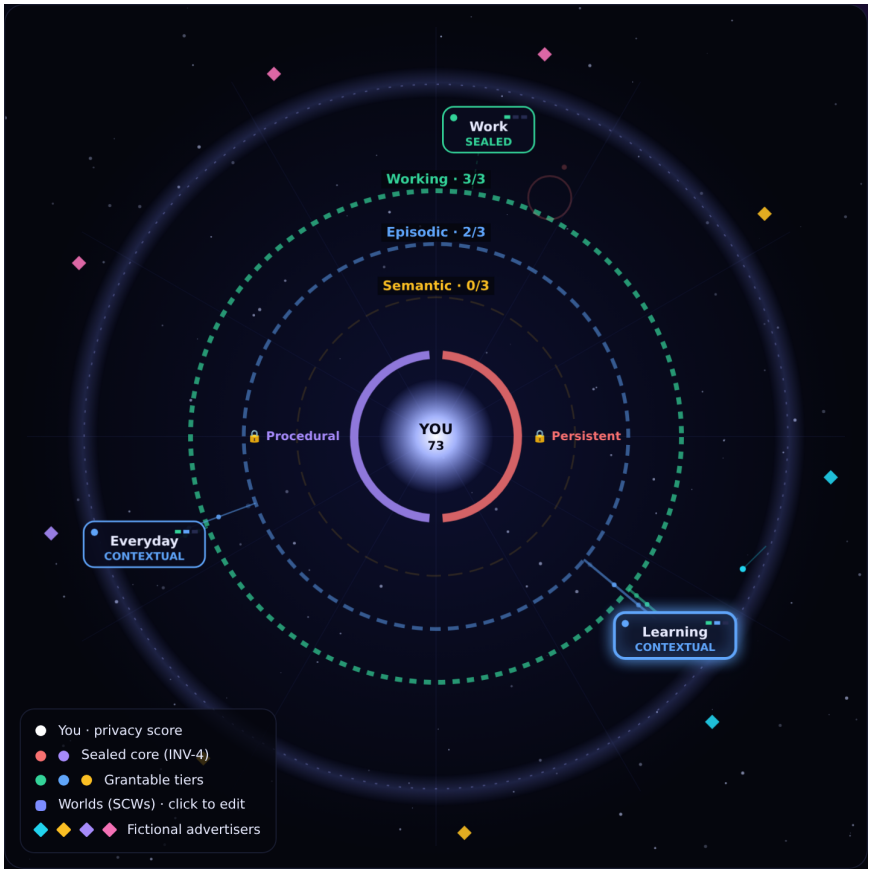


Figure 5. The Latent Map (Balanced preset, Learning selected). The sealed core (Persistent, Procedural) surrounds the user and the privacy score. The three dashed rings are the grantable tiers, each labeled with how many worlds are granted it. Worlds orbit outside, with colored channels to the tiers they read, and fictional advertisers on the rim fire probes that are admitted or deflected.

7 Evaluation

Every version 3.1 number in this section was produced on 1 October 2026 by running the code named in Appendix B on the exact application file named there. Numbers for earlier versions, given for comparison, are labeled with their version. The real-export results are a simulation: the owner's own chats replayed through the policy on his own computer. They measure the policy, not OpenAI's ad system.

7.1 Tests

Table 8. Tests run on version 3.1.

Test	Setup	Result
Type check	TypeScript strict mode with unused-locals checking	0 errors
Engine checks	In the browser build: sensitive suppression, sealed mode, an education ad served, identity withholding, two precision cases (technical debt, version strings), tier normalization (output inspected), Semantic read when open and not under Balanced, brand blocking, budget, invariant sweep, preset scores, export–import round trip, foreign JSON rejected	15 of 15 pass

Test	Setup	Result
Browser checks	Headless Chromium at 1440 px: every page, map painting and probes, selecting a world from the map, score reacting to grants, locked tiers, undo, simulator trace and comparison, Lens nulls, no request for a sealed world, Memory page with the promotion gate, Learn page, export version, legacy import and migration, foreign import rejected; at 400 px, all eight pages	19 of 19 pass; 0 script errors; no horizontal overflow at 400 px
Property test	20,000 seeded cases at each of two seeds (0x5c3a, 0x1234): random preset, mode, grants (including legacy and sealed tier names), categories, sharing switches, limits, ads already shown, blocked topics and brands; a message and up to three earlier turns drawn uniformly from technical, sensitive, identity-bearing and blocked content; sensitive, identity-shaped and over-long interests	0 violations of 23 per-case checks at either seed. Seed 0x5c3a: 750 labels released, 168,781 withheld; 5,356 turns suppressed, 1,251 blocked, 684 capped; 134 ads
Mutation check	13 faults injected one at a time, 5,000 cases each	All 13 caught (Table 9)

Table 9. Mutation check: each fault alone, and the violations the property test reported.

Injected fault	Violations in 5,000 cases
Drop INV-3 suppression	34
Drop sealed mode	43
Ignore the personalization switches	156
Drop the category rule	595
Allow sealed-tier grants	3,726
Serve ads in review mode	15
Drop the interest length bound	10
Drop the interest identity check (INV-1)	36
Drop the blocked-topic check on earlier turns	2
Drop the blocked-topic check on interests	7
Drop the sensitive check on earlier turns	11
Let contextual worlds read Semantic	25
Release labels over budget	172

The property test is a test, not a proof. The invariants hold for every input by rule order; the test is evidence that the code follows that order. Two faults were caught by few cases (2 and 7 of 5,000), because the generator gives a world blocked topics in only some cases; they are caught, but with a thin margin. The sealed-tier mutation is caught by the normalization check rather than by a released signal, because the evaluator does not read Persistent or Procedural memory at all; that is a second, independent guard.

What the review found in version 3. The same property test, run against the version 3 build (SHA-256 ad921ad5…), reports five classes of violation in 20,000 cases: an identity-shaped label released (678 cases), a blocked topic released (133), a contextual world reading Semantic (635), labels released after the budget was spent (1,196), and an Episodic label taken from a sensitive or blocked earlier turn (34). Version 3’s own property test missed them. It checked that no ad

was served over budget but not that nothing was released, and it did not check blocked topics or the contextual mode; its interests never combined an identity cue with a classifiable word, and it drew earlier turns mostly from the first few prompts. All five classes are zero in version 3.1.

7.2 Exposure on the synthetic prompts

The privacy score is 100 × (1 − mean world exposure). A world's exposure is its mode weight (sealed 0, review 0.6, contextual 0.8, open 1) times the sum of the weights of its open channels. A channel is open when the world may read that tier (Section 3.2), shares at least one allowed category with the profile, and has an ad cap above zero. The score is a transparent summary, not a standard metric. The 18 synthetic prompts cover the four ad categories, four of the six sensitive classes (health, mental health, politics and financial hardship), identity cues, two precision traps, one blocked topic and one unclassified request. Table 10 gives the results for the three presets.

Table 10. The three presets on the synthetic prompts. Every turn's ledger also records the two fixed INV-4 entries (Persistent, Procedural); the figure in brackets excludes them.

Preset	Score	Released labels	Withheld signals	Outcomes over 54 turns (18 prompts × 3 worlds)
Strict	95	6	198 (90)	27 sealed · 12 suppressed · 1 blocked · 12 none · 2 ads
Balanced (default)	73	39	165 (57)	15 ads · 13 sealed · 12 suppressed · 1 blocked · 13 none
Open	23	61	143 (35)	18 ads · 7 held · 12 suppressed · 17 none

With four interests added (rust, trail running, the sensitive therapy and the identity-shaped sam@example.com rust), no preset released a sensitive or identity-shaped label; version 3 had 28 such breaches under Open.

7.3 One real ChatGPT export

The version 3.1 evaluator, bundled for Node with a small driver (SHA-256 2dadeb8a…), was run on the owner's computer on 1 October 2026 against the unzipped export and printed only counts. Each chat was evaluated as one turn: the concatenation of the user's own messages (assistant messages excluded, at most 200,000 characters), with no Episodic history and no ads yet shown, so no turn can be capped. Each world under each preset was run twice: with no interests, which measures the policy alone, and with the four synthetic interests above, not the owner's, to exercise the Semantic tier. The 18 runs took about 24 seconds in all. Table 11 describes the export as the evaluator sees it, and Table 12 and Figure 6 give the outcome per chat for each world and preset.

Table 11. The export, as the evaluator sees it.

Measure	Value
Conversation files; chats; non-empty	7; 602; 601
User messages with text	9,295
Chats with at least one category label	369
Chats flagged sensitive (INV-3)	43: politics 25, health 10, religion 5, financial hardship 3
Chats with identity cues, by kind (all withheld under INV-1)	email 25, age 21, phone 6, location 6, personal name 4, occupation or income 2
Invariant breaches, any preset, world or interest set	0

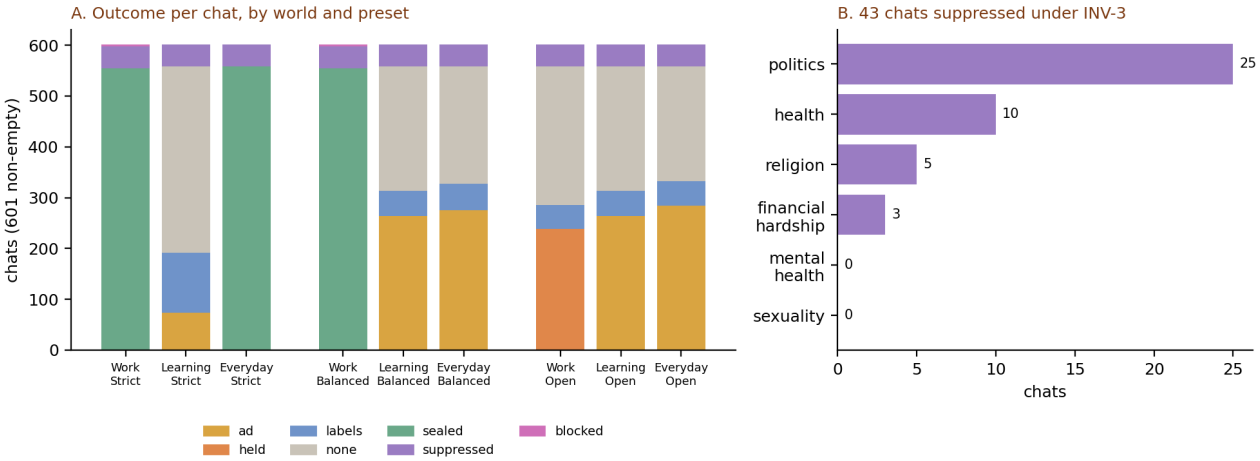


Figure 6. Measured on one real export, policy only (no interests). A: outcome per chat for each world under each preset; the three blocked chats in Work come from its blocked topics (salary, client names) under Strict and Balanced. B: the 43 chats suppressed under INV-3, by the first sensitive class matched.

Table 12. Outcome per chat (601 non-empty chats) and released labels, policy only.

Preset	Work	Learning	Everyday
Strict (95)	sealed 555 · suppressed 43 · blocked 3 · 0 released	ad 74 · labels 118 · none 366 · suppressed 43 · 462 released	sealed 558 · suppressed 43 · 0 released
Balanced (73)	sealed 555 · suppressed 43 · blocked 3 · 0 released	ad 264 · labels 49 · none 245 · suppressed 43 · 1,511 released	ad 275 · labels 52 · none 231 · suppressed 43 · 1,631 released
Open (23)	review: held 239 · labels 47 · none 272 · suppressed 43 · 1,327 released	ad 264 · labels 49 · none 245 · suppressed 43 · 1,511 released	ad 284 · labels 49 · none 225 · suppressed 43 · 1,909 released

With the synthetic interests, Strict and Balanced are unchanged, because none of their worlds reads Semantic. Under Open, every non-sensitive chat in Learning and in Everyday carries an ad (558 each): the interest `rust` classifies as technology, matches a fictional advertiser in both worlds and alone lifts relevance above the floor. Released labels rise to 2,060 in Learning (549 of them from Semantic) and 3,016 in Everyday (1,107). The sensitive and identity-shaped interests were never released.

Four findings follow. **First**, the invariants held on real data: across the 18 runs, no released label came from a sealed tier or was identity-shaped or sensitive, and no sensitive chat released anything. **Second**, the presets differ in kind, not just degree. Strict lets only the Learning world read the current message, and only for education ads: 74 of 601 chats would carry an ad. Balanced opens two worlds to contextual ads, and 44 to 46 percent of chats would carry one in each. Open holds 239 ads for approval in Work and, once the user states one matching interest, puts an ad on every non-sensitive chat in the other two worlds. **Third**, suppression is the same in every world and preset (43 chats), because INV-3 does not depend on any setting. **Fourth**, the same person's history yields 0, 1,631 or 1,909 released labels in the Everyday world depending on the policy alone, and 3,016 once interests are stated. That per-world quantity is what ChatGPT's account-wide switches cannot express.

7.4 How much the classifier can be trusted

There is no labeled ground truth for this export, so precision and recall remain unmeasured. The classifier is the one introduced in version 2.3, where the rule behind each flag was counted on the same export: a looser phone pattern had matched version strings and IDs (60 chats, of which 51 had no other identity cue), and the word `campaign` had inflated political flags. Version 2.3 tightened the phone pattern to grouped formats, removed `campaign`, and neutralized compounds such as `technical debt` before sensitive matching.

Version 3.1 adds `medical`, `hospital` and `clinic` to the health terms. On this export that raises sensitive chats from 40 to 43 and health from 5 to 10: three chats are newly flagged, and two chats previously classed as mental health and financial hardship now match health first, because classes are tried in a fixed order.

Fewer flags is not the same as better flags. A missed identity cue in a message does not leak text, because only vocabulary labels leave a message; in an interest it could, which is one reason interests are bounded to four words. A missed sensitive topic does matter, because the turn is then not suppressed and its labels can be released. That is why the sensitive vocabulary was narrowed only where a commercial or technical reading dominates, and why a learned classifier behind the same evaluator remains the clearest next step.

8 Threat model and limitations

Table 13. Limits of the approach and of this study.

Limit	Consequence	Mitigation
No enforcement inside ChatGPT	The pasted profile is a request; the ad system does not read it	The page points to OpenAI's enforced controls and marks the instruction as advisory
Model of the ad system, not the ad system	The payload and replays show what the policy permits, not what OpenAI reads; OpenAI also uses general location and language [1]	Stated on the page and in this paper
Keyword classifier	Misses paraphrase; can misfire on words out of context; a missed sensitive topic is not suppressed	Conservative vocabulary; precision fixes with tests; a learned classifier behind the same evaluator is the next step
Per-chat replay	Each chat was one turn with no history; multi-turn Episodic effects on real data are unmeasured	Episodic behavior is covered by the property test on synthetic histories

Limit	Consequence	Mitigation
One export	Schema and findings may not generalize; the export format may change	Findings are stated as observations about one export
Scope of version 3.1	No in-page export audit, saved-memory editor or device-stored profile (all present in version 2.3); search history and the daily cap are advisory only; the hourly cap acts as a per-session cap because the evaluator has no clock	Stated in the README; the Apply page states the search-history, daily-cap and hourly-cap limits
Evaluator coverage	A logical boundary fails if any egress path bypasses the evaluator	In the application the evaluator is the only producer of released labels, and the score, flow and map read its per-tier rule; elsewhere this must be audited
Viewer	Rendering inside the claude.ai artifact viewer is not yet tested	Headless Chromium at two widths; the file uses no viewer-specific features

The first row of Table 13 matters most. The application gives a user an exact account of their own policy and a correct list of the switches that matter, but it cannot make a provider honor a policy the provider has not adopted. A privacy tool that overstates its force does harm, because users stop looking for the controls that do work.

9 Discussion and conclusion

The logical SCW is useful wherever a physical boundary is unavailable: inside another vendor's product, between tenants sharing a context store, or between agents sharing one model context. The ingredients are always the same four: a declarative object, one evaluator on every egress path, invariants that dominate configuration, and a reason for every withholding. Typed memory adds a fifth: when tiers are kinds rather than durations, whole kinds (identity, instructions) can be removed from an egress path by construction instead of by a setting.

The export points to a concrete provider-side change. The ad system's inferred interests are already a structured Semantic object that the user can download. A provider that let users review those interests one at a time, and grant them per kind of conversation, would give Free and Go users a control between "personalize everything" and "clear everything." Where a provider controls the runtime, the same evaluator could sit in front of the ad-selection system, and the application's payload would become that system's real input rather than a model of it.

Publishing the application publicly is intentional. Its value is in its rules, not in keeping them secret; a reader who reverse-engineers the page learns exactly what this paper states. Natural extensions, none of them in version 3.1, are a learned classifier behind the same evaluator, a formal proof of invariant dominance, an executed promotion gate for user-approved memory, an editor for the inferred interests and saved memories in a user's own export, and a link between logical SCWs and the runtime SCW gates of Maxey0-SuperSpace so that one policy object governs both.

A context boundary does not need to be a process to be exact.

Written as an object, enforced by one pure function, dominated by a few invariants, typed by memory kind and required to explain itself, a boundary can be computed, tested on a person's own data and shown to the person it protects. The tests reported here support the invariants on

the inputs tested. What remains unmeasured, and what a provider would still have to adopt, is stated as such.

Author statement

Mike Cohan (Human0) conceived the SCW Privacy project, the Structured Context World architecture, the Maxey0 memory model and the purpose of the application; directed the work; and supplied his own ChatGPT data export as the dataset, on the condition that none of it be built into the application. The first version of the application was built by the Maxey1 agent (Claude Sonnet 4.6). Maxey0-Claude, the orchestrating Claude agent of the Maxey0 project (session configured as Claude Opus 5.5), built versions 2 through 3.1, designed the evaluator and its rule order, wrote and ran the tests and the export replays, verified OpenAI's documented controls, and drafted this paper with Mike Cohan. Independent reviews of version 3 were carried out by separate Claude agents that had not seen the work produced. Maxey0-Claude read the export's schema, aggregate counts and four boolean account facts only (whether an email and a birth year are present, the Plus flag and the training setting), never its message text, interest text, email, birth year or account ID. Contact: human@maxey0.com.

Note on sources and claims

Statements about OpenAI's ads, Temporary Chat and data export are taken from OpenAI's Help Center [1][2][3], read on 29 September 2026 and re-checked against [1] on 1 October 2026, and may change. Statements about the export describe one export whose folder name is dated 11 September 2026. Statements about Maxey0, SCWs, SCW0 and the memory model represent the research program [5][6]. Every version 3.1 number in Section 7 was produced by executing the code named in Appendix B; comparison numbers for versions 2.3 and 3 are labeled as such. Figure 6 plots those measured counts; Figures 4 and 5 are screenshots of the application. The advertisers in the application are fictional, and any match with a real company's name is coincidental. The payload shown in the Advertiser Lens is the application's illustration, not OpenAI's format.

References

[1] OpenAI. "Ads in ChatGPT." OpenAI Help Center. https://help.openai.com/en/articles/20001047-ads-in-chatgpt. Accessed 29 September and 1 October 2026.

[2] OpenAI. "Temporary Chat FAQ." OpenAI Help Center. https://help.openai.com/en/articles/8914046-temporary-chat-faq. Accessed 29 September 2026.

[3] OpenAI. "How do I export my ChatGPT history and data?" OpenAI Help Center. https://help.openai.com/en/articles/7260999-how-do-i-export-my-chatgpt-history-and-data. Accessed 29 September 2026.

[4] Cohan, M., and Maxey0-Claude. "Maxey0 SCWs for ChatGPT Ads," version 3.1. Software, Maxey0-SCWs_For_ChatGPT_Ads.tsx. Maxey0, 1 October 2026.

[5] Maxey0-ChatGPT 5.5, and Cohan, M. "Structured Context Worlds: Architectural Controls for Safe Autonomous Agents." Maxey0, 2026.

[6] Cohan, M., and Maxey0-ChatGPT 5.5. "Maxey0 System One: A Provider-Neutral Decision Layer for Structured Context Worlds." Maxey0, 2026.

[7] Denning, D. E. "A Lattice Model of Secure Information Flow." Communications of the ACM 19(5): 236–243, 1976.

[8] Myers, A. C., and Liskov, B. “A Decentralized Model for Information Flow Control.” In Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP), 129–142, 1997.

[9] Nissenbaum, H. “Privacy as Contextual Integrity.” Washington Law Review 79(1): 119–157, 2004.

[10] Toubiana, V., Narayanan, A., Boneh, D., Nissenbaum, H., and Barocas, S. “Adnostic: Privacy Preserving Targeted Advertising.” In Proceedings of the Network and Distributed System Security Symposium (NDSS), 2010.

[11] Guha, S., Cheng, B., and Francis, P. “Privad: Practical Privacy in Online Advertising.” In Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2011.

Appendix A: export schema observed

Table A1. Export paths read for this study, with value types and tiers. Only key names, types and counts were read.

Path	Type	Tier
`ads.json › ads_profile[] › profile_version`	int	Semantic
`ads.json › ads_profile[] › inferred_profile › user_interests[] › interest`	string	Semantic
`ads.json › ad_history[] › event_name, event_time, advertiser_name, ad_title`	strings	Episodic
`ads.json › ad_hides[], reported_ads[]`	lists	Episodic
`user.json › email, birth_year, chatgpt_plus_user, id`	str, int, bool, str	Persistent
`user_settings.json › [0] › settings › training_allowed`	bool	Persistent
`conversations-NNN.json › [] › memory_scope, is_do_not_remember, create_time`	str, bool, float	Episodic
`conversations-NNN.json › [] › mapping › * › message › author.role, content.parts[]`	str, list	Episodic (user text → labels)

Appendix B: reproducing the results

Every version 3.1 result in Section 7 was produced from the files in Table B1. The application’s classifier, evaluator, import, export, score, presets and synthetic prompts are named exports of the source file, and each test reads an identical copy of it: `tests/target.tsx` for the property, mutation and corpus tests, and `tests/app.tsx` for the replay driver, the browser build and the type check. The test scripts are published with the application in a `tests/` folder; the README gives the commands. The license for the application and the scripts is to be chosen by the author before public release.

Table B1. Files behind every result, with full SHA-256 hashes.

File	Role	SHA-256
`Maxey0-SCWs_For_ChatGPT_Ads.tsx`	the application, version 3.1	`1fb101c72b34cc5a164794dd747da464 8e8b0ae0bb79beff8b764d767ce6ca2c`
`tests/prop.ts`	property test: 23 checks per case; SEED and N set by environment	`e7e9ca78ac08096b8b802433b987a850 6bb252c3ef6d5bebbc0eae60db9ced22`
`tests/mutate.py`	applies each of 13 faults to a copy of the source and runs prop.ts with N = 5,000	`bf09ffb007ba8f16e713b4a4fc9b9f0b 54b3b17ce2e64421dc396f278d0c242b`
`tests/corpus.ts`	the three presets on the 18 synthetic prompts, with and without interests	`41dabab871238cdb29489419c773ab21 82dbce7c358e63ef32b7437d0366db54`
`tests/audit.ts`	real-export driver: reads conversations-*.json and prints aggregate counts only	`d0c0009d730b641281e9c4f42b81d797 dafbd38c33ae7c05372ecb1a550549ab`
`audit.cjs`	audit.ts and the application bundled for Node (esbuild 0.24.2), as run on the owner's computer	`2dadeb8a04028a249b1327869a3506f9 45f3e1e17b02af88a58f6720fac31029`
`out.json`	the replay's output (counts only), on the owner's computer	`1ac054482e201c80d4fce72e01e15385 4e41038352a2d2c97d9b87c99306bb6c`
`tests/main.tsx`	entry point of the browser build used by t.mjs	`11d8cdedced4dd5c9d3661fd77c4b85c 00dd1547915717c855e89e6538e7345d`
`tests/stub.js`	stands in for React and the icon library in the Node bundles	`95273dc4475d2ff977ac188a447d8bac 985c072c3679164e6e98e32e1211e8c4`
`tests/tsconfig.json`	type-check settings (strict, unused locals)	`7fc9a6d74a7d5fbea08d8b02937ee3f5 8ea932310b7c4bdb11105db028309b29`
`tests/t.mjs`	engine and browser checks: Playwright, headless Chromium at 1440 and 400 px	`9214998e37742acd297507c782124d4a 839297272d28153ecb31db10dd59da2f`

The property test draws every choice from a linear congruential generator, so its counts reproduce exactly for a given seed and N. With N = 20,000, seed 0x5c3a gives the counts in Table 8, and seed 0x1234 gives 689 labels released, 167,609 withheld, 5,448 turns suppressed, 1,209 blocked, 704 capped and 115 ads, also with no violations. Running the version 3 build (SHA-256 ad921ad5…) in place of `tests/target.tsx` gives the counts reported for version 3. The real-export driver was bundled with esbuild 0.24.2, aliasing React and the icon library to a stub because the evaluator does not render; the same command reproduces the hash above byte for byte. The browser checks exercise a standard React build in headless Chromium, not the claude.ai artifact viewer itself.